

DAY 4

FastAPI - Response Models, Status Codes & Path Operation Configuration

1. Response Model - Return Type

Annotating a path operation function's return type (for example, `-> Item`) makes FastAPI validate the returned data, filter it to the declared fields, serialize it via Pydantic, and add it to the OpenAPI schema/docs.

response_model parameter

Use **response_model** on the path operation decorator when the function returns something different, such as a dict or database object, but you still want FastAPI to treat it as a Pydantic model for docs, validation, and filtering.

Note: If both a return type and `response_model` are declared, `response_model` takes priority.

```
@app.post("/items/", response_model=Item)
async def create_item(item: Item) -> Any:
    return item
```

Filtering out sensitive fields

Declare a **UserIn** model with the password for input and a separate **UserOut** model without the password for the response, so sensitive data does not leak.

Inheritance trick

Define **BaseUser** with public fields, then **UserIn(BaseUser)** with a password. Annotate the function as `-> BaseUser` while returning a **UserIn** instance. FastAPI filters the response down to **BaseUser**'s fields.

Returning a Response directly

If the return type is **Response** or a subclass such as **RedirectResponse** or **JSONResponse**, FastAPI skips response model generation automatically. A union such as **Response | dict** fails; use `response_model=None` to disable response model generation.

response_model_exclude_unset

`response_model_exclude_unset=True` includes only fields explicitly set on the object, omitting values that only came from defaults. Related options are `response_model_exclude_defaults=True` and `response_model_exclude_none=True`.

response_model_include / response_model_exclude

These are shortcuts for including or excluding named fields. Separate model classes are preferred because OpenAPI can still expose the full model schema.

2. Response Status Code

Set the HTTP status code with the **status_code** parameter on the path operation decorator.

```
@app.post("/items/", status_code=201)
async def create_item(name: str):
    return {"name": name}
```

HTTP status code ranges

- 100-199: Informational - rarely used directly; no response body allowed.
- 200-299: Success - 200 OK, 201 Created, 204 No Content.
- 300-399: Redirection - may or may not have a body; 304 must not.
- 400-499: Client error - for example 404 Not Found.
- 500-599: Server error - usually returned automatically for internal failures.

Note: Prefer constants such as `status.HTTP_201_CREATED` instead of memorizing numeric codes.

3. Path Operation Configuration

Metadata parameters can be passed directly to the path operation decorator to configure how an endpoint appears in OpenAPI/docs.

- **status_code** - sets the response status.
- **tags** - groups endpoints in the docs UI.
- **summary** and **description** - short and long endpoint descriptions.
- **Docstring** - used as the description if no explicit description is supplied; Markdown is supported.
- **response_description** - describes the response specifically.
- **deprecated=True** - marks the endpoint as deprecated without removing it.

```
@app.post(
    "/items/",
    tags=["items"],
    summary="Create an item",
    response_description="The created item",
)
async def create_item(item: Item) -> Item:
    """Create an item with all the info."""
    return item
```

Source: fastapi.tiangolo.com/tutorial/response-model, [/response-status-code](#), [/path-operation-configuration](#)